Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 32

Graphs

Representation & Exploration Template

Graphs

- A graph is a representation of binary *relations*
  - A set of entities V and a relation $E \subseteq V \times V$ that contains all pairs (u,v) such that "u is related to v"
  - An element in E is called an *edge*, and it "connects" a pair of *nodes* in V
- E.g., Webpage u points to webpage v; profile u is a friend of profile v; point u is connected to point v by a road; game state u can evolve into game state v in one step; package u must be installed before installing package v; etc.
- Several graph algorithms and data structures enable a large number of applications that require answering questions about large graphs quickly

Graphs

- A *path* in a graph is a sequence of distinct nodes u_0, u_1, ..., u_k such that every pair $(u_i,u_{i+1}) \in E$
- Note that in general, $(u,v) \in E$ doesn't necessarily mean $(v,u) \in E$ (e.g., package dependencies)
  - But in some graphs, we have $(u,v) \in E \Leftrightarrow (v,u) \in E$. They are symmetric or *undirected* graphs (unless otherwise stated, our graphs are directed)
- In some applications, the edges have a "weight": e.g., the distance of a road, network latency or bandwidth, probability of state transition, etc.

Representing Graphs

- Basic requirements from a data structure storing a graph
  - Enumerate outgoing edges: Given a node u, enumerate all nodes in its *neighbourhood*: i.e., all v s.t. (u,v) ∈ E
    - Less often: Enumerate incoming edges
  - Given a candidate edge (u,v), check if (u,v) ∈ E
  - Add/delete a node/edge
- Nodes are typically labeled {0,...,n-1}
  - If labeled by other keys, can use an auxiliary hash table to map keys to indices
- There are different data structures with different trade-offs in time, space and features
  - E.g., if fast neighbourhood enumeration not required, can store all (u,v) pairs in a standard set data structure

Attendance Break!

Login using CSE LDAP id at

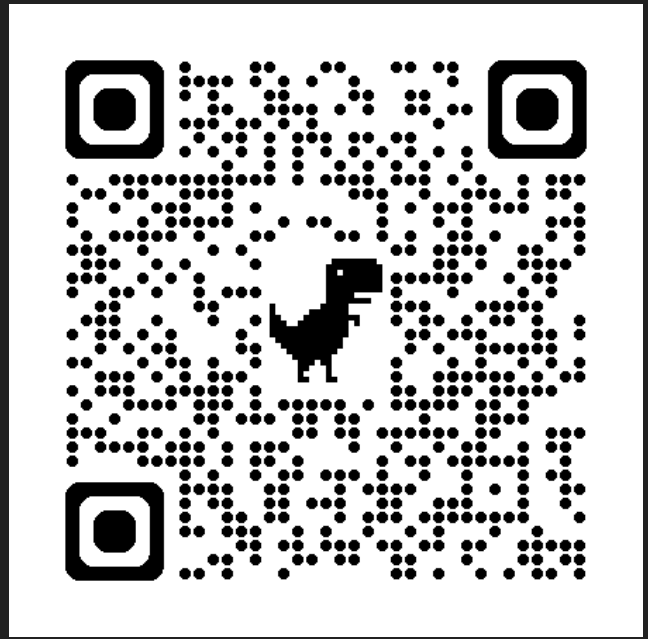
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

Honour Code: Don't leave the lecture after marking attendance!

If you can't attend the whole lecture don't mark attendance!

If you are not in the classroom, a report will be sent to DADAC!



Adjacency Matrix

- V = {0,..,n-1}. Store E as an n×n 2D-matrix, of 0s and 1s
  - `A[u][v] == 0` means $(u,v) \notin E$, `A[u][v] == 1` means $(u,v) \in E$
  - Implemented as an array where $(u,v)^{th}$ index is at location $u \cdot n+v$
- Can instantly check if $(u,v) \in E$
- Can quickly enumerate outgoing edges: $\Theta(n)$ time, but elements will be in cache
  - Can enumerate incoming edges too (but less cache friendly)
- Main drawback: $\Theta(n^2)$ space. Not suitable for large, sparse graphs
  - Somewhat mitigated by packing the matrix entries as bits (`std::bitset<64>` stores 64 bits in one int rather than in 64 bools). Enables exploiting parallelisation of certain operations (e.g., intersection or union of neighbourhoods)

Adjacency List

- Store E as an array of lists: A[u] has a list of all v s.t. $(u,v) \in E$
  - Total space used is $\Theta(m+n)$ rather than $\Theta(n^2)$, where $|V|=n$, $|E|=m$
- To check if a given $(u,v) \in E$, check the list stored in A[u]
- Suited only for sparse graphs where all nodes have small degree
- The "list" of neighbours should be stored as a vector, not a linked-list
  - More cache-friendly, and no overhead from the links
  - Can add a neighbour using push_back in O(1) time. Can delete a neighbour (after having found it) in O(1) time by popping the last one and writing it back on top of the neighbour to be deleted.

Adjacency Map

- When using an adjacency list, if a few nodes have high degree, looking up their neighbours is slow
  - In many real-life applications, graphs have widely varying degrees
- Idea: For such large-degree nodes, instead of a list (vector), use a *set* data structure
  - Nodes with small degrees will still use a vector, to be more cache friendly
  - If neighbourhood needs to be explored in sorted order, use a balanced BST; otherwise a hash table gives O(1) amortised access time (w.h.p.)

Compressed Sparse Row

- Recall adjacency list representation: A[u] has a vector of all neighbours of u
  - To be more cache-friendly, we can concatenate all these vectors into a single vector, `nbrs[0,..,m-1]` (m is the total number of edges)
  - Will maintain boundaries of the original vectors in `offset[0,..,n]`
    - Neighbours of node u in `nbrs[offset[u],..,offset[u+1]-1]`
    - If u has no outgoing edges, empty range: `offset[u]==offset[u+1]`,
    - `offset[n]` is set to `m`
  - Like adjacency list, not good for high-degree nodes. Further, not suited for addition/deletion of edges.

Exploring a Graph

- In many applications using graphs, important to be able to start at a "source node" and discover all nodes reachable from s
- A general template

```
// frontier: a container for nodes; discovered[v] false for all nodes v
frontier.insert(s); discovered[s] = true;
while (!frontier.empty()) {
  u = frontier.extract();           // then may process u as necessary
  for ( v : graph.neighbours(u) ) {
    if(discovered[v]) continue;
    frontier.insert(v); discovered[v] = true;
  }
}
```

- Main variation: Which node will be extracted from frontier (LIFO/FIFO/...)

Exploring a Graph

- No matter what container is used for frontier, as long as extraction on a non-empty container removes and returns some element, the algorithm terminates (every node is inserted into frontier at most once, and in each iteration one node is extracted), and the set of nodes discovered $s=u_0, u_1, \ldots, u_t$ are exactly all the nodes reachable from s
- Any node discovered is reachable
  - Inductively, u_k is reachable using nodes $u_0,\ldots,u_{k-1}$: u_k was discovered from some node u_i, $i<k$, and inductively there is a path from s to u_i; add to this path the new edge (u_i,u_k) to get a valid path to u_k
- Any reachable node is discovered
  - Suppose there is a path $s=v_0, v_1, \ldots, v_t=v$, but v is not discovered. Let v_i $(i>0)$ be the first undiscovered node in this path. So v_{i-1} was discovered and added to the frontier. Before the frontier becomes empty, v_{i-1} must be extracted. But then v_i will be discovered!